



JURUSAN TEKNOLOGI INFORMASI

Mata Kuliah Basis Data Lanjut

## 05. Index dan Optimasi Query



# Topik



1. Konsep Dasar Index
2. Analisis Performa Query
3. Membuat dan Mengelola Index
4. Optimasi Query dengan Index
5. Latihan Dasar-dasar Index & Penggunaannya
6. Latihan Optimasi Query & Trade-off Index

# *Topik-1: Konsep Dasar Index*

---

---

# 1. Konsep Dasar Index

- **Index** di database adalah **struktur data tambahan** yang dibuat di atas tabel untuk mempercepat pencarian data.
- Kalau tabel adalah **buku tebal**, maka index adalah **daftar isi / kamus indeks** yang memberitahu kita langsung ke halaman tertentu tanpa harus membaca seluruh isi buku.
  - Tanpa index → PostgreSQL harus membaca seluruh baris (**Sequential Scan**) untuk menemukan data.
  - Dengan index → PostgreSQL cukup mencari di struktur index, lalu langsung menuju baris yang relevan (**Index Scan**).
- Mengapa Penting?
  - **Kecepatan Akses**
    - Query yang tadinya butuh **detik-menit** bisa dipangkas jadi **milidetik**.
    - Misalnya, mencari pelanggan dengan email tertentu dari **50.000 baris** → tanpa index PostgreSQL cek satu per satu, dengan index bisa langsung loncat.
  - **Efisiensi Resource**
    - Mengurangi CPU dan I/O.
    - Membuat query lebih hemat resource → penting kalau data besar.
  - **Mendukung Operasi Kompleks.** Index juga dipakai untuk:
    - Sorting (ORDER BY).
    - Pencarian teks (full-text search, trigram).
    - Query spasial (lokasi, koordinat).
    - JOIN antar tabel besar.

# 1. Konsep Dasar Index Cara Kerja

- Misalnya query ini:

```
SELECT * FROM pelanggan WHERE email = 'pelanggan12345@mail.com';
```

- **Tanpa Index**

- PostgreSQL melakukan **Sequential Scan**: cek baris 1, baris 2, baris 3 ... sampai ketemu.
- Untuk **50.000 baris**, worst-case butuh baca semua → lambat.

- **Dengan Index (B-Tree di email):**

```
CREATE INDEX idx_pelanggan_email ON pelanggan(email);
```

- PostgreSQL menyimpan struktur data **B-Tree** di belakang layar.
- Email pelanggan sudah **terurut** di pohon ini.
- PostgreSQL tinggal menelusuri pohon,  **$\log_2(N)$**  langkah.
- Untuk 50.000 baris, hanya butuh  $\pm 16$  langkah → jauh lebih cepat.
- Analogi sederhana:
  - **Tanpa Index** → seperti mencari kata “Indexing” di **buku 1000 halaman** dengan membaca halaman 1–1000.
  - **Dengan Index** → buka daftar isi → langsung ke halaman 356.

# 1. Konsep Dasar Index

## Jenis Index



1. **B-Tree (default)** → cocok untuk  $=$ ,  $<$ ,  $>$ , BETWEEN.
2. **Hash** → khusus  $=$  (lebih jarang dipakai karena B-Tree sudah bagus).
3. **GIN** → cocok untuk **full-text search** atau kolom array.
4. **GiST** → untuk data spasial, similarity search, dll.
5. **BRIN** → hemat ruang, cocok untuk data besar dengan nilai terurut (misalnya tanggal log).

# 1. Konsep Dasar Index

## Jenis Index #1: B-Tree Index (Default)

- **Cara kerja:**
  - PostgreSQL membuat struktur data **balanced tree (B-Tree)**.
  - Data disimpan dalam urutan yang terurut, sehingga pencarian bisa dilakukan cepat dengan logaritmik ( $O(\log n)$ ).
- **Kegunaan:**
  - Cocok untuk pencarian **equality** ( $=$ ,  $<>$ ).
  - Cocok untuk pencarian **range** ( $<$ ,  $>$ , BETWEEN).
  - Bisa mempercepat ORDER BY dan DISTINCT.

- **Contoh penggunaan:**

```
CREATE INDEX idx_pelanggan_email ON pelanggan(email);  
  
SELECT * FROM pelanggan WHERE email = 'pelanggan12345@mail.com';  
SELECT * FROM transaksi WHERE tanggal_transaksi BETWEEN '2024-01-01' AND '2024-06-30';
```

- **Catatan:**
  - Ini adalah **index default** di PostgreSQL.
  - Hampir semua kebutuhan umum cukup dengan B-Tree.

# 1. Konsep Dasar Index

## Jenis Index #2: Hash Index

- **Cara kerja:**
  - Menggunakan **hashing function** untuk membuat nilai hash dari kolom.
  - Pencarian jadi cepat, tapi hanya bisa untuk **equality check (=)**.
- **Kegunaan:**
  - Sangat cocok untuk WHERE kolom = nilai.
  - Tidak bisa dipakai untuk range (<, >, BETWEEN).

- **Contoh penggunaan:**

```
CREATE INDEX idx_pelanggan_email_hash ON pelanggan USING hash(email);  
  
SELECT * FROM pelanggan WHERE email = 'pelanggan12345@mail.com';
```

- **Catatan:**
  - Dulu (versi lama PostgreSQL), Hash Index tidak WAL-logged → tidak aman crash.
  - Sejak PostgreSQL 10 → sudah WAL-logged, aman digunakan.
  - Tetapi: **B-Tree sering lebih dipilih**, karena sudah cukup cepat & fleksibel.

## 1. Konsep Dasar Index

### Jenis Index #3 – GIN (Generalized Inverted Index)

- **Cara kerja:**

- Menyimpan **inverted index** (seperti search engine).
- Mengindex **kata/token** dalam teks atau elemen dalam array.
- Bisa menangani **banyak nilai dalam satu kolom** (multi-value indexing).

- **Kegunaan:**

- **Full-text search** (to\_tsvector, to\_tsquery).
- **Array search** (@>, <@, &&).
- JSONB (indexing key & value di JSON dokumen).

- **Contoh penggunaan (Full-text Search):**

```
CREATE INDEX idx_produk_deskripsi_gin ON produk USING  
gin(to_tsvector('english', deskripsi));
```

```
SELECT * FROM produk WHERE to_tsvector('english', deskripsi) @@  
to_tsquery('laptop & gaming');
```

- **Contoh penggunaan (Array):**

```
CREATE INDEX idx_tags_gin ON artikel USING gin(tags);
```

```
SELECT * FROM artikel WHERE tags @> ARRAY['database'];
```

- **Catatan:**

- Cocok kalau satu kolom berisi **teks panjang** atau **array banyak nilai**.
- Bisa membuat query ribuan kali lebih cepat dibanding sequential scan.

## 1. Konsep Dasar Index

### Jenis Index #4: - Generalized Search Tree

- **Cara kerja:**
  - Framework index yang fleksibel → bisa dipakai untuk banyak tipe data.
  - Cocok untuk data yang butuh **similarity search** atau **range multidimensi**.
- **Kegunaan:**
  - **Data spasial/geometris** (misalnya koordinat, peta, GIS).
  - **Full-text search alternatif** (misalnya trigram search).
  - **Similarity search** (misalnya mencari string mirip).
- **Catatan:**
  - Sering dipakai bersama **PostGIS** untuk query geospasial.
  - Lebih fleksibel daripada GIN, tapi kadang lebih lambat untuk pencarian teks sederhana.

- **Contoh penggunaan (Geometri):**

```
CREATE INDEX idx_lokasi_gist
ON lokasi USING gist(geom);

SELECT * FROM lokasi
WHERE ST_DWithin(geom, ST_MakePoint(112.75, -7.25),
1000);
```

- **Contoh penggunaan (Trigram Search):**

```
CREATE EXTENSION pg_trgm;

CREATE INDEX idx_produk_nama_trgm
ON produk USING gist(nama_produk gist_trgm_ops);

-- typo search
SELECT * FROM produk WHERE nama_produk % 'laptopp';
```

## 1. Konsep Dasar Index

### Jenis Index #5: BRIN (Block Range Index)

- **Cara kerja:**
  - Tidak menyimpan pointer ke setiap row, tapi hanya menyimpan **summary per block** (rentang nilai min-max).
  - Sangat hemat ruang → index bisa ribuan kali lebih kecil dibanding B-Tree.
- **Kegunaan:**
  - **Data besar yang terurut secara alami** → misalnya tanggal, id, atau data log.
  - Cocok untuk **append-only tables** (misalnya data sensor, transaksi harian, log server).
- **Contoh penggunaan:**

```
CREATE INDEX idx_transaksi_tanggal_brin
ON transaksi USING brin(tanggal_transaksi);

SELECT * FROM transaksi
WHERE tanggal_transaksi BETWEEN '2024-01-01' AND '2024-06-30';
```

- **Kelebihan:**
  - Sangat kecil dan cepat dibuat.
  - Ideal untuk tabel **ratusan juta – miliaran baris**.
- **Kekurangan:**
  - Kurang akurat dibanding B-Tree.
  - PostgreSQL tetap mungkin membaca block yang tidak relevan.

# 1. Konsep Dasar Index

## Ringkasan Jenis Index



| No. | Jenis Index   | Cocok Untuk                          | Ukuran       | Catatan                           |
|-----|---------------|--------------------------------------|--------------|-----------------------------------|
| 1   | <b>B-Tree</b> | Equality & Range                     | Sedang       | Default, paling sering            |
| 2   | <b>Hash</b>   | Equality                             | Sedang       | Hanya =                           |
| 3   | <b>GIN</b>    | Full-text, Array, JSONB              | Besar        | Search engine-like                |
| 4   | <b>GiST</b>   | Spasial, Similarity, Trigram         | Sedang–besar | Fleksibel, dipakai di PostGIS     |
| 5   | <b>BRIN</b>   | Data besar & terurut (misal tanggal) | Sangat kecil | Cocok untuk tabel >100 juta baris |

# 1. Konsep Dasar Index

## Perbandingan dengan MySQL

- **Index default:**
  - **PostgreSQL**
    - **B-Tree** adalah index default pada sintaksis CREATE INDEX.
    - Cocok untuk operasi umum (=, <, >, BETWEEN, ORDER BY).
  - **MySQL (InnoDB)**
    - **B-Tree** juga default, tapi implementasi di MySQL hanya untuk **equality dan range** (kurang fleksibel dibanding PostgreSQL).
    - Index InnoDB sekaligus menyimpan data row di **clustered index (PRIMARY KEY)**.
- **Jenis index yang di-support:**

| Jenis Index      | PostgreSQL                               | MySQL                                                                          |
|------------------|------------------------------------------|--------------------------------------------------------------------------------|
| <b>B-Tree</b>    | ✓ Default, sangat fleksibel              | ✓ Default, tapi terbatas pada equality & range                                 |
| <b>Hash</b>      | ✓ Native sejak v10                       | ⚠ Tidak didukung di InnoDB, hanya di Memory Engine (jarang dipakai)            |
| <b>GIN</b>       | ✓ Full-text search, array, JSONB         | ⚠ Tidak ada. Full-text pakai index B-Tree khusus di MyISAM/InnoDB              |
| <b>GiST</b>      | ✓ Spasial, trigram, similarity           | ⚠ Tidak ada. Spasial pakai <b>SPATIAL index</b> (MyISAM dulu, sekarang InnoDB) |
| <b>BRIN</b>      | ✓ Hemat ruang untuk data besar & terurut | ✗ Tidak ada padanan langsung                                                   |
| <b>Full-Text</b> | ✓ Dengan GIN/GiST                        | ✓ Native di InnoDB (sejak 5.6), hanya untuk teks                               |

## 1. Konsep Dasar Index

### Perbandingan dengan MySQL – Secara Umum...

- PostgreSQL unggul di variasi jenis index (GIN, GiST, BRIN, JSONB, array, full-text canggih).
- MySQL lebih terbatas: mayoritas hanya **B-Tree + FULLTEXT + SPATIAL**, tapi dengan kelebihan **clustered index default** di InnoDB.
- Jika ingin fokus ke **analisis data, full-text, GIS, JSON**, PostgreSQL lebih kaya.
- Jika fokusnya hanya ke **aplikasi OLTP standar**, MySQL cukup dengan B-Tree.

## *Topik-2: Analisis Performa Query*

---

---

## 2. Analisis Performa Query

- **Analisis Performa Query** adalah proses mengevaluasi **bagaimana database menjalankan sebuah query (SQL)**, untuk mengetahui:
  - **Apakah query efisien atau lambat.**
  - **Mengapa lambat** (misalnya: table scan, join besar, sorting tanpa index).
  - **Bagaimana memperbaikinya** (misalnya dengan index, rewrite query, denormalisasi).
- Dengan kata lain: ini adalah cara kita **mengintip dapur PostgreSQL** → melihat rencana eksekusi query dan biaya tiap langkah.
- **Mengapa Penting?**
  - **Dataset besar = resiko query lambat:** Query yang kelihatan sederhana (SELECT \* FROM transaksi WHERE id\_pelanggan=123) bisa butuh **detik/menit** kalau tabelnya jutaan baris tanpa index.
  - **Optimasi sumber daya:** Database server punya keterbatasan CPU, RAM, I/O. Query lambat = resource habis. Analisis performa membantu kita menghemat resource.
  - **Memastikan penggunaan index:** Kadang kita sudah buat index, tapi query tetap lambat → analisis performa bisa menunjukkan kalau PostgreSQL masih pakai **Sequential Scan** (index tidak terpakai).
  - **Mendeteksi bottleneck:** Apakah lambat karena JOIN? Sorting (ORDER BY)? Agregasi (GROUP BY)? Filtering?
- Cara melakukan:
  - Gunakan fitur **EXPLAIN**, **EXPLAIN ANALYZE**, dan **pg\_stat\_statements**.

## 2. Analisis Performa Query EXPLAIN



- EXPLAIN adalah perintah di PostgreSQL untuk **melihat rencana eksekusi query** tanpa benar-benar menjalankan query tersebut. Dengan EXPLAIN kita bisa:
  - Membantu **mendeteksi query lambat**.
  - Menunjukkan apakah **index sudah terpakai**.
  - Memberi gambaran **biaya relatif (cost)** tiap langkah eksekusi
- Dengan EXPLAIN, kita bisa tahu **bagaimana PostgreSQL berniat mengambil data**:
  - Apakah lewat **Sequential Scan** (baca semua row)?
  - Atau lewat **Index Scan** (pakai index)?
  - Bagaimana urutan operasi JOIN, FILTER, SORT?
- Contoh:

```
EXPLAIN SELECT * FROM pelanggan WHERE email = 'pelanggan12345@mail.com';
```
- Hasil:

```
Seq Scan on pelanggan (cost=0.00..1150.00 rows=1 width=100)
  Filter: (email = 'pelanggan12345@mail.com')
```
- Kalau sudah ada index pada "email", maka hasilnya akan berubah menjadi:

```
Index Scan using idx_pelanggan_email on pelanggan (cost=0.42..8.44 rows=1 width=100)
  Index Cond: (email = 'pelanggan12345@mail.com')
```
- Sekarang PostgreSQL pakai **Index Scan**, jauh lebih efisien.

## 2. Analisis Performa Query

### EXPLAIN ANALYZE

- EXPLAIN ANALYZE adalah perintah untuk **menjalankan query sungguhan** dan sekaligus menampilkan **rencana eksekusi + waktu eksekusi nyata** dari tiap langkah.
  - Jika EXPLAIN hanya simulasi, maka EXPLAIN ANALYZE benar-benar mengeksekusi query-nya.
  - EXPLAIN bisa:
    - Memberi tahu **berapa lama query berjalan** (dalam milidetik).
    - Membandingkan **estimasi PostgreSQL** (cost, rows) dengan **kenyataan**.
    - Membantu menemukan bottleneck, misalnya sorting atau join yang makan waktu besar.
- Contoh:**

```
EXPLAIN ANALYZE SELECT * FROM pelanggan WHERE email =  
'pelanggan12345@mail.com';
```
- Hasil**

```
Index Scan using idx_pelanggan_email on pelanggan  
  (cost=0.42..8.44 rows=1 width=100)  
  (actual time=0.050..0.060 rows=1 loops=1)  
Planning Time: 0.070 ms  
Execution Time: 0.120 ms
```
- Interpretasi:**
  - PostgreSQL pakai **Index Scan**.
  - Estimasi rows=1, kenyataan rows=1 (sesuai).
  - Query selesai dalam **0.120 ms**.

## 2. Analisis Performa Query **pg\_stat\_statements**



- **pg\_stat\_statements** adalah **extension PostgreSQL** yang merekam **statistik eksekusi query** di server.
- Dengan ini, kita bisa tahu:
  - Query apa saja yang paling sering dipakai.
  - Query mana yang paling lambat atau paling berat.
  - Rata-rata waktu eksekusi tiap query.
- **Kenapa Penting?**
  - Membantu **monitoring performa database**.
  - Bisa dipakai untuk **menemukan query yang jadi bottleneck**.
  - Sangat berguna di sistem produksi, di mana ada ribuan query berbeda.

## 2. Analisis Performa Query `pg_stat_statements`

- Cara menggunakan:

```
-- aktifkan extension (sekali saja per database)
CREATE EXTENSION pg_stat_statements;

-- lihat 10 query paling berat
SELECT query, calls, total_exec_time, mean_exec_time
FROM pg_stat_statements
ORDER BY total_exec_time DESC
LIMIT 10;
```

- Contoh Output:

| query                               | calls | total_exec_time | mean_exec_time |
|-------------------------------------|-------|-----------------|----------------|
| SELECT * FROM transaksi WHERE id=.. | 5000  | 12000 ms        | 2.4 ms         |
| INSERT INTO transaksi ...           | 2000  | 8000 ms         | 4.0 ms         |

- Dari tabel ini kita tahu query mana yang **paling banyak makan waktu**.

## *Topik-3: Membuat dan Mengelola Index*

---

---

### 3. Membuat dan Mengelola Index

#### Sintaksis dasar CREATE INDEX

- Index dibuat dengan perintah CREATE INDEX. Index ini akan membantu PostgreSQL mencari data lebih cepat, karena PostgreSQL tidak perlu membaca seluruh baris (sequential scan), melainkan langsung menuju baris yang relevan menggunakan index.
- Contoh sintaks dasar:

```
CREATE INDEX nama_index ON nama_tabel(nama_kolom);
```
- Misalnya, jika kita sering mencari pelanggan berdasarkan email, maka kita buat index di kolom tersebut:

```
CREATE INDEX idx_pelanggan_email ON pelanggan(email);
```
- Dengan index ini, query seperti:

```
SELECT * FROM pelanggan WHERE email = 'pelanggan123@mail.com';
```
- akan lebih cepat dieksekusi karena PostgreSQL langsung menggunakan index untuk menemukan baris yang sesuai.

### 3. Membuat dan Mengelola Index

## Index tunggal vs index multi-kolom

- **Index tunggal** dibuat hanya pada satu kolom. Cocok jika query biasanya memfilter dengan satu kolom tertentu.

```
CREATE INDEX idx_transaksi_tanggal ON transaksi(tanggal_transaksi);
```

- Query seperti, berikut ini akan lebih cepat dieksekusi:

```
SELECT * FROM transaksi WHERE tanggal_transaksi = '2024-12-01';
```

- **Index multi-kolom** dibuat pada lebih dari satu kolom. Ini berguna jika query sering menggunakan kondisi kombinasi dari dua atau lebih kolom sekaligus.

```
CREATE INDEX idx_transaksi_pelanggan_tanggal ON transaksi(id_pelanggan, tanggal_transaksi);
```

- Query seperti berikut ini akan lebih cepat karena PostgreSQL menggunakan index gabungan tadi.

```
SELECT * FROM transaksi WHERE id_pelanggan = 123 AND tanggal_transaksi BETWEEN '2024-01-01' AND '2024-12-31';
```

- Perlu diingat: index multi-kolom hanya efisien jika urutan kolom dalam index sesuai dengan pola query yang digunakan.

### 3. Membuat dan Mengelola Index

## Unique Index & Kaitannya dengan Constraint

- **Unique Index** memastikan bahwa nilai dalam kolom tertentu tidak boleh duplikat. Misalnya, email pelanggan harus unik.
- Cara membuat unique index:

```
CREATE UNIQUE INDEX idx_pelanggan_email_unique ON pelanggan(email);
```

- Namun, biasanya kita tidak membuat unique index secara manual, melainkan menggunakan **constraint**:

```
ALTER TABLE pelanggan ADD CONSTRAINT pelanggan_email_unique UNIQUE (email);
```

- Kedua perintah tersebut sebenarnya akan menghasilkan hal yang sama: PostgreSQL membuat index unik di belakang layar.
  - Bedanya, constraint lebih bersifat deklaratif dan lebih jelas secara semantik (bahwa kolom ini memang harus unik sebagai bagian dari aturan data).

### 3. Membuat dan Mengelola Index Partial Index

- **Partial Index** adalah index yang hanya mencakup baris tertentu sesuai kondisi WHERE. Ini berguna ketika kita sering melakukan query pada subset data tertentu, bukan seluruh tabel.
- Contoh:

```
CREATE INDEX idx_transaksi_recent
ON transaksi(tanggal_transaksi)
WHERE tanggal_transaksi > CURRENT_DATE - INTERVAL '30 days';
```
- Dengan index ini, PostgreSQL akan membuat index hanya untuk transaksi 30 hari terakhir. Jadi, query:

```
SELECT * FROM transaksi WHERE tanggal_transaksi >
CURRENT_DATE - INTERVAL '30 days';
```
- akan sangat cepat karena PostgreSQL cukup melihat subset index, bukan seluruh tabel.
- Kelebihan partial index adalah ukurannya jauh lebih kecil dibanding index penuh.

### 3. Membuat dan Mengelola Index

## Menghapus Index (DROP INDEX)

- Jika sebuah index sudah tidak diperlukan lagi, kita bisa menghapusnya untuk menghemat ruang penyimpanan dan mengurangi beban saat operasi tulis.

- Contoh:

```
DROP INDEX idx_pelanggan_email;
```

- Setelah index dihapus, query-query yang sebelumnya cepat karena memanfaatkan index mungkin akan kembali lambat, karena PostgreSQL harus melakukan sequential scan.

### 3. Membuat dan Mengelola Index

## Dampak Negatif Index

- Walaupun index sangat berguna untuk mempercepat SELECT, kita harus menyadari ada beberapa dampak negatif atau trade-off:
  1. **Menghabiskan ruang penyimpanan tambahan**
    - Setiap index adalah struktur data terpisah. Jika kita punya banyak index, ukuran database bisa membengkak.
  2. **Memperlambat operasi tulis (INSERT, UPDATE, DELETE)**
    - Saat ada baris baru dimasukkan atau data diubah, PostgreSQL tidak hanya memperbarui tabel, tapi juga semua index yang relevan. Semakin banyak index, semakin lambat operasi tulis.
  3. **Beban pada optimizer**
    - Jika terlalu banyak index, PostgreSQL harus memilih index mana yang paling optimal. Kadang ini justru membuat optimizer salah memilih strategi.
- Karena itu, prinsipnya adalah **buat index hanya jika benar-benar dibutuhkan oleh query yang sering dijalankan**. Index yang berlebihan tidak hanya membuang ruang, tapi juga bisa memperlambat sistem.

# *Topik-4: Optimasi Query dengan Index*

---

---

## 4. Optimasi Query dengan Index

- Optimasi query adalah upaya untuk membuat perintah SQL berjalan seefisien mungkin, baik dari segi waktu maupun pemakaian sumber daya (CPU, RAM, dan I/O disk).
  - Index adalah salah satu alat utama dalam optimasi, namun penggunaannya harus hati-hati karena index juga punya biaya (*cost*) tambahan. **Tidak selalu kita harus membuat index.**
- **Index perlu dibuat ketika:**
  - Kolom sering dipakai di kondisi **WHERE**.
    - Contoh: kolom email pelanggan, atau kolom tanggal\_transaksi.
  - Kolom sering dipakai untuk **JOIN**.
    - Contoh: id\_pelanggan pada tabel transaksi.
  - Kolom sering dipakai di **ORDER BY** atau **GROUP BY**.
    - Index bisa membantu mempercepat sorting dan grouping.
  - Query sering **dijalankan dan lambat tanpa index**.
    - Dengan index, query bisa dipercepat berlipat-lipat.
- **Index TIDAK perlu dibuat ketika:**
  - Tabel berisi data kecil (misalnya hanya beberapa ratus baris). Sequential scan kadang lebih cepat daripada pakai index.
  - Kolom jarang dipakai di query.
  - Kolom memiliki sedikit variasi nilai (low selectivity).
    - Misalnya kolom gender hanya berisi L atau P. Index tidak banyak membantu karena PostgreSQL tetap harus membaca hampir semua baris.
  - Tabel sangat sering di-*update* atau di-*insert*.
    - Banyak index justru akan memperlambat operasi tulis, karena setiap perubahan data harus memperbarui index juga.

## 4. Optimasi Query dengan Index

### Strategi Optimasi Query #1: Menulis Query yang Ramah Index

- Kadang query yang salah ditulis bisa membuat index tidak digunakan, meskipun index tersedia.
  - PostgreSQL hanya bisa memanfaatkan index jika kondisi WHERE ditulis langsung pada kolom, tanpa transformasi atau fungsi tambahan.
- **Contoh yang buruk (index tidak digunakan):**

```
SELECT * FROM pelanggan WHERE LOWER(email) = 'pelanggan123@mail.com';
```

- Meskipun kolom email sudah diindex, PostgreSQL tidak bisa memakainya karena ada fungsi LOWER() di kolom tersebut.
  - LOWER() harus dihitung untuk setiap baris, sehingga index menjadi tidak relevan.
- **Solusi yang benar:**
  - Simpan data dengan format konsisten (misalnya selalu lowercase).
  - Atau buat **functional index**:

```
CREATE INDEX idx_pelanggan_email_lower ON pelanggan(LOWER(email));
```

- Setelah itu, query dengan LOWER(email) bisa memakai index.
  - Intinya: **hindari fungsi atau operasi langsung di kolom jika ingin index bekerja.**

## 4. Optimasi Query dengan Index

### Strategi Optimasi Query #2: Gunakan ANALYZE untuk update statistik tabel

- PostgreSQL punya **query optimizer** yang memilih apakah akan menggunakan index atau tidak. Optimizer ini bekerja berdasarkan **statistik tabel** (jumlah baris, distribusi nilai, dsb).
  - Jika statistik sudah lama tidak diperbarui, PostgreSQL bisa salah mengambil keputusan.
- Misalnya, kita sudah membuat index di email, tapi query tetap lambat karena PostgreSQL salah mengira distribusi data.
  - Solusinya adalah menjalankan ANALYZE untuk memperbarui statistik:

```
ANALYZE pelanggan;
```

- Biasanya PostgreSQL melakukan ini otomatis lewat autovacuum, tapi untuk tabel yang sering berubah besar, kita bisa melakukannya manual.
  - Dengan statistik yang up-to-date, PostgreSQL bisa memutuskan strategi eksekusi query yang paling efisien.

## 4. Optimasi Query dengan Index

### Strategi Optimasi Query #3: Query rewrite (misalnya IN vs JOIN)

- Cara menulis query bisa memengaruhi performa, meskipun hasil akhirnya sama.
  - Ada kalanya PostgreSQL bisa mengoptimasi query lebih baik jika ditulis dengan cara tertentu.

- **Contoh dengan IN:**

```
SELECT * FROM transaksi
WHERE id_pelanggan IN (SELECT id_pelanggan FROM pelanggan
WHERE kota = 'Jakarta');
```

- **Alternatif dengan JOIN:**

```
SELECT t.*
FROM transaksi t
JOIN pelanggan p ON t.id_pelanggan = p.id_pelanggan
WHERE p.kota = 'Jakarta';
```

- Di beberapa kasus, query dengan JOIN lebih efisien karena PostgreSQL bisa menggunakan index di kedua tabel sekaligus.
  - Sedangkan query dengan IN kadang menghasilkan subquery yang sulit dioptimasi.
  - Hal yang sama juga berlaku untuk query lain: kadang menulis ulang query (query rewrite) bisa membuat PostgreSQL memakai index dengan lebih baik.

## 4. Optimasi Query dengan Index

### Ringkasan



- Index harus digunakan dengan bijak: buat index hanya jika query memang membutuhkan.
- Query harus ditulis dengan cara yang ramah index, misalnya tanpa fungsi langsung di kolom, kecuali sudah dibuat functional index.
- Statistik tabel harus diperbarui dengan ANALYZE, supaya optimizer PostgreSQL memilih strategi yang tepat.
- Query rewrite, seperti mengganti IN dengan JOIN, bisa membuat PostgreSQL lebih efisien dalam menggunakan index.
- Dengan memahami kapan membuat index, bagaimana menulis query yang ramah index, dan bagaimana optimizer bekerja, kita bisa memastikan performa query tetap cepat meskipun data sudah sangat besar.

# *Topik-5: Latihan Dasar-dasar Index & Penggunaannya*

---

---

## 5. Latihan Dasar-dasar Index & Penggunaannya

- Penjelasan topik ini ada di **Petunjuk Praktikum**.

# *Topik-6: Latihan Optimasi Query & Trade-off Index*

---

---

## 6. Latihan Optimasi Query & Trade-off Index

- Penjelasan topik ini ada di **Petunjuk Praktikum**.

# Pertanyaan?



*Terima Kasih*

# Tugas



- Selesaikan langkah-langkah pada petunjuk praktikum.

# Referensi



[1] -